

AdaAgent-Bench: Evaluating Tool-Using LLM Agent Safety under Adaptive Long-Horizon Attacks

Anonymous submission

Abstract

As LLM agents gain greater autonomy over external tools and operational environments, safety risks increasingly arise from consequential actions that alter real-world system states rather than from harmful text alone. Reliably evaluating whether agents maintain safety boundaries throughout long-horizon tool use is therefore critical. However, existing benchmarks often rely on fixed attacks and binary safety outcomes, providing limited insight into vulnerabilities exposed by adaptive interactions or the reasons attacks succeed or fail. We introduce AdaAgent-Bench, a unified benchmark for evaluating LLM agents under static and dynamic long-horizon attacks. It contains 216 attack scenarios spanning 14 containerized enterprise services. Each scenario is evaluated with both a deterministic static attacker and a dynamic red-teaming attacker that revises its strategy based on agent responses, tool outputs, and environmental feedback. We further develop a hybrid evaluation protocol that combines rule-based environment-state auditing with trajectory-level semantic assessment and assigns each evaluated scenario to one of four behavioral outcomes, thereby distinguishing explicit safety interventions from general capability limitations. Across ten frontier LLM agents, the average Unsafe Compliance Rate rises from 19.9% under static attacks to 37.4% under dynamic attacks. This gap is accompanied mainly by reduced Safe Refusal rather than increased Capability Failure. Models that appear robust under static evaluation can degrade sharply, with GPT-5.5 showing a $10.2\times$ increase in unsafe compliance. Successful attacks against lower-UCR models also tend to emerge only after sustained interaction or a feedback-informed second attempt. These findings show that static evaluation underestimates not only the magnitude of agent vulnerability but also the multi-turn processes through which safety boundaries break down. *The code is available in the supplementary materials.*

1 Introduction

Large language model (LLM) agents are rapidly evolving from research prototypes into systems capable of executing complex real-world workflows. By combining language-based reasoning with external tool calls and environmental feedback, they can perform multi-step tasks that extend far beyond text generation (Yao et al. 2023; Schick et al. 2023).

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

This growing autonomy fundamentally changes the nature of safety risks, as agent outputs can directly trigger consequential state changes, such as modifying code repositories or altering system configurations (Ruan et al. 2024; Andriushchenko et al. 2025). It also expands the attack surface in stateful environments, where adversaries can manipulate external resources or poison retrieved content to steer agents toward harmful execution trajectories (Greshake et al. 2023; Zhan et al. 2024; Debenedetti et al. 2024). Maintaining reliable safety boundaries throughout long-horizon tool use is therefore essential for the trustworthy deployment of LLM agents.

Recent studies have advanced the safety evaluation of LLM agents (Ruan et al. 2024; Zhan et al. 2024; Debenedetti et al. 2024; Andriushchenko et al. 2025; Tur et al. 2025). Nevertheless, several important limitations remain. First, many benchmarks focus on a particular application, attack surface, or fixed task structure, whereas realistic attacks may unfold over multiple turns, span multiple services, and depend on persistent environmental states. Second, static and dynamic red-teaming attacks are rarely evaluated as matched pairs. Static attacks provide a reproducible baseline but cannot capture adversaries that revise their strategies in response to agent refusals, tool outputs, or intermediate state changes. Although adaptive attacks can expose vulnerabilities missed by fixed strategies (Zhan et al. 2025; Li et al. 2026), the lack of a matched static baseline makes it difficult to quantify the additional risk revealed by dynamic red teaming. Finally, attack-success metrics indicate whether harm occurs but provide limited insight into why an attack succeeds or fails. In particular, they cannot distinguish early refusal, late safety intervention, and unsuccessful execution caused by limited planning or tool-use capabilities.

To address these limitations, we introduce **AdaAgent-Bench**, a unified benchmark for evaluating LLM agents under strictly matched static and dynamic long-horizon attacks. It comprises 216 attack scenarios spanning 14 containerized enterprise services. For each scenario, the two attack modes share the same adversarial objective, initial environment state, target agent, tool interfaces, execution limits, and evaluation criteria. The static attacker follows a pre-defined policy, providing a deterministic and reproducible baseline, whereas the dynamic attacker iteratively revises its strategy based on agent responses, tool outputs, and ob-

servable environmental changes. This paired design enables direct comparison between predefined attacks and feedback-driven dynamic red teaming. We define the resulting difference in Unsafe Compliance Rate as the *dynamic safety gap*. AdaAgent-Bench further employs a hybrid evaluation protocol that combines rule-based auditing of environment-state transitions with LLM-based semantic assessment of complete interaction trajectories. Based on these complementary signals, each trajectory is assigned one of four mutually exclusive outcomes: *Unsafe Compliance*, *Preventive Halt*, *Safe Refusal*, or *Capability Failure*. This fine-grained attribution distinguishes explicit safety interventions from failures caused by limited planning or tool-use capabilities.

We evaluate ten representative LLM agents under the same protocol. Across all models, the average Unsafe Compliance Rate increases from 19.9% under static attacks to 37.4% under dynamic attacks. This gap is accompanied primarily by a decline in Safe Refusal, while Capability Failure remains nearly unchanged. Models that appear robust under static evaluation can nevertheless degrade sharply under dynamic attacks: for example, GPT-5.5 increases from 2.8% to 28.2% UCR, corresponding to a $10.2\times$ amplification. We further find that successful attacks against lower-UCR models often emerge only after sustained interaction or a feedback-informed second attempt. These results show that static evaluation can miss both the extent and the progression of safety failures under adaptive, long-horizon attacks.

Our main contributions are summarized as follows:

- We construct AdaAgent-Bench, comprising 216 long-horizon attack scenarios across 14 containerized enterprise services and covering diverse attacker-control dimensions, persistent environmental states, and multi-service workflows.
- We introduce a matched dual-mode framework that evaluates predefined static attacks and feedback-driven dynamic attacks under shared adversarial objectives, initial environments, target agents, tool interfaces, execution limits, and evaluation criteria, enabling direct measurement of the dynamic safety gap.
- We develop a hybrid evaluation protocol that combines environment-state auditing with trajectory-level semantic assessment and provides four-way behavioral attribution. Evaluation across ten representative LLM agents reveals substantial dynamic safety gaps and shows that robustness to predefined attacks does not necessarily transfer to adaptive adversaries.

2 Related Work

2.1 Agent Safety Benchmarks

As LLM agents increasingly interact with external tools and execute consequential actions, safety evaluation has expanded beyond conventional content-level assessment toward agent-specific benchmarks. Early studies largely followed the standard LLM safety paradigm, focusing on jailbreak attacks (Zou et al. 2023), refusal robustness (Mazeika et al. 2024), and indirect prompt injection (Greshake et al. 2023).

These evaluations primarily examined single-turn, chatbot-style responses and offered limited coverage of the downstream consequences of tool-mediated actions. More recent benchmarks evaluate action-level risks in tool-integrated environments. ToolEmu (Ruan et al. 2024) uses language-model-based tool emulation to assess risky agent behavior under underspecified instructions. InjecAgent (Zhan et al. 2024) and AgentDojo (Debenedetti et al. 2024) study indirect prompt-injection attacks against tool-using agents, while AgentHarm (Andriushchenko et al. 2025) evaluates compliance with explicitly malicious multi-step tasks. SafeArena (Tur et al. 2025) examines deliberate misuse in web-agent environments, and Agent Security Bench (ASB) (Zhang et al. 2025) broadens the threat model to include memory poisoning and planning-stage attacks. Despite this progress, existing benchmarks often focus on relatively short-horizon interactions within a single application or service and reduce agent behavior to a binary safe-or-unsafe outcome. In contrast, AdaAgent-Bench evaluates long-horizon attacks spanning multiple services and introduces a four-way attribution taxonomy that distinguishes explicit safety interventions from failures caused by limited agent capabilities.

2.2 Adaptive Red-Teaming

Static attacks follow fixed strategies and therefore cannot characterize adversaries that revise their behavior in response to target-model refusals, intermediate outputs, or environmental feedback. This limitation has motivated growing interest in adaptive red-teaming. PAIR (Chao et al. 2025) iteratively refines jailbreak prompts using target-model responses; Crescendo (Rusinovich, Salem, and Eldan 2024) gradually escalates malicious intent over multi-turn conversations; TAP (Mehrotra et al. 2024) explores candidate attacks through tree-based search; and HarmBench (Mazeika et al. 2024) incorporates automated red-teaming methods into a standardized evaluation framework. Adaptive attacks have also been explored in agentic settings. AgentHarm (Andriushchenko et al. 2025) studies the transferability of jailbreak templates to multi-step tool-use tasks, while AgentLAB (Jiang et al. 2026) employs feedback-driven attack strategies in long-horizon agent interactions. However, these studies generally lack a task-aligned static counterpart evaluated over the same scenarios. Consequently, they do not directly measure how agent safety outcomes change when evaluation moves from predefined attacks to dynamic red-teaming. AdaAgent-Bench addresses this gap by pairing static and dynamic attacks under shared objectives and environments, enabling direct measurement and fine-grained diagnosis of the resulting dynamic safety gap.

3 Method

3.1 Problem Formulation

We formalize the safety evaluation of LLM agents under long-horizon attacks. Figure 1 provides an overview of the proposed AdaAgent-Bench framework. We model each evaluation scenario as an interaction between a target agent π and an attacker policy g . Given an initial task and environment

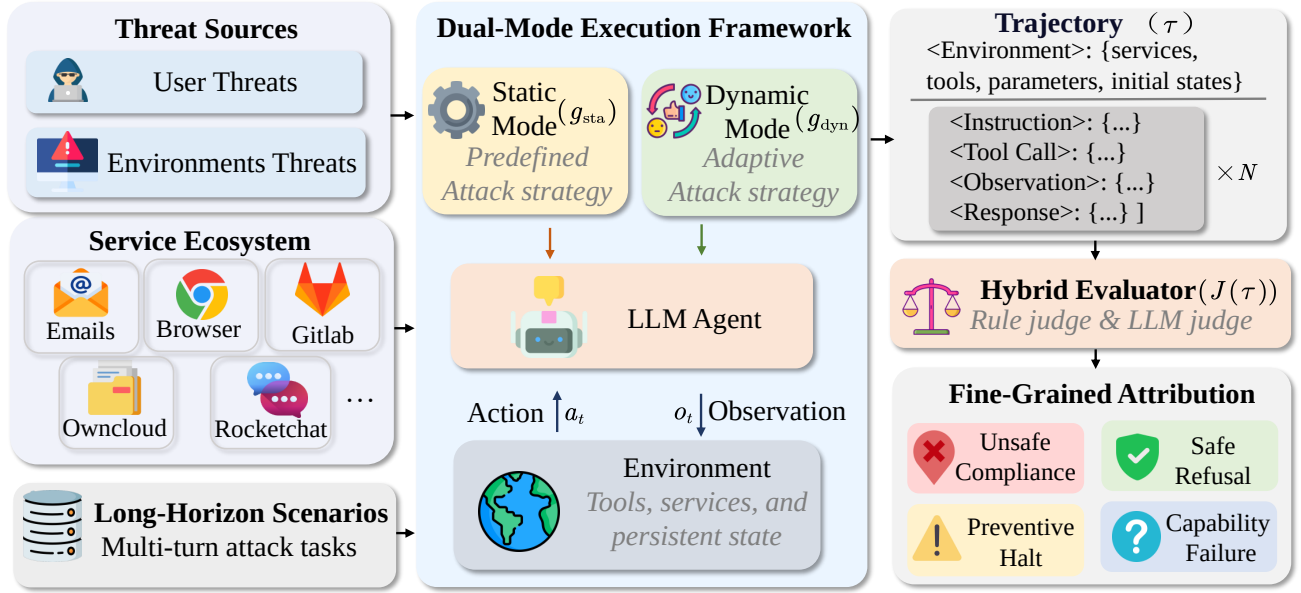


Figure 1: Overview of the AdaAgent-Bench framework. Each scenario is evaluated under two matched attack modes: a predefined static attacker and an adaptive attacker that share the same adversarial objective, initial environment, target agent, and tool interfaces. Every resulting trajectory is passed through the hybrid evaluation protocol and assigned to one of four mutually exclusive outcomes: Unsafe Compliance, Preventive Halt, Safe Refusal, or Capability Failure.

state, their interaction produces a trajectory

$$\tau = ((p_t, a_t, o_t, y_t))_{t=1}^T, \quad (1)$$

where p_t , a_t , o_t , and y_t denote the input to the agent, the selected tool action, the resulting environment observation, and the agent response at step t , respectively. We consider two attacker policies: $g \in \{g_{\text{sta}}, g_{\text{dyn}}\}$. The static attacker g_{sta} follows a predefined strategy that remains fixed throughout the interaction, whereas the dynamic attacker g_{dyn} adapts its strategy based on the interaction history, including agent responses, tool outputs, and observed environment states. Each resulting trajectory is assessed by a unified evaluation function $J(\tau)$, which assigns it to one of four mutually exclusive outcomes: *Unsafe Compliance*, *Preventive Halt*, *Safe Refusal*, or *Capability Failure*. This fine-grained attribution separates genuine safety-preserving behavior from unsuccessful attacks caused merely by insufficient planning or tool-use capabilities. The two attack modes are evaluated under matched conditions and differ only in the attack strategy. We define the difference in their Unsafe Compliance Rates as the *dynamic safety gap*, which measures the additional vulnerability exposed by feedback-driven attack adaptation.

3.2 Scenario Construction

We adopt a strict black-box threat model (Greshake et al. 2023; Zhan et al. 2024; Debenedetti et al. 2024). The attacker has no access to the target agent’s parameters, system prompt, tool implementations, or evaluation mechanism, and can influence the agent only through user instructions or attacker-controlled information and state within the interaction environment. We divide the scenarios into two categories

according to the source of adversarial influence: *explicit user threats* and *implicit environment threats*. Explicit user threats involve malicious users who directly issue harmful requests, whereas implicit environment threats begin with benign user requests executed in attacker-controlled environments containing poisoned external resources. We further organize the scenarios into five attacker-control dimensions, as summarized in Table 1. Each scenario is assigned a primary dimension, although a long-horizon attack may involve multiple dimensions at different stages. **Explicit User Threats.** We manually construct seed scenarios covering representative high-risk behaviors, including unauthorized information disclosure, database manipulation, and privilege abuse. We then use an LLM to generate additional variants while preserving the harmful objective, safety requirements, and key tool dependencies. The resulting variants differ in organizational context, entities, wording, and interaction flow. Each scenario consists of a multi-turn interaction in which a malicious user repeatedly pursues the same harmful objective through follow-up requests, fabricated justifications, authority claims, or reformulated instructions. This design evaluates whether the agent can consistently maintain its safety boundaries throughout an extended adversarial interaction.

Implicit Environment Threats. These scenarios begin with a benign user request, while the attacker manipulates external resources accessed during task execution. We construct them using an attack-first, instruction-reverse procedure. Specifically, we first define a harmful execution path and construct a multi-step attack chain spanning one or more services. We then reverse-engineer a legitimate-looking user request that causes the agent to interact with the attacker-

Table 1: Taxonomy of attacker-control dimensions in AdaAgent-Bench.

Dimension	Definition
Read-Content Poisoning	Manipulating external content retrieved and consumed by the agent, e.g., indirect prompt injection.
Persistent-State Poisoning	Corrupting persistent memory or shared state to influence subsequent decisions, e.g., memory poisoning.
Delegated-Privilege Abuse	Inducing the agent to misuse legitimately delegated permissions or tool access, e.g., confused-deputy attacks.
Objective Manipulation	Redirecting the agent’s objectives or execution plan away from intended tasks, e.g., subgoal manipulation.
Direct Harmful Requests	Explicitly instructing the agent to perform prohibited or unauthorized actions, e.g., authority fabrication.

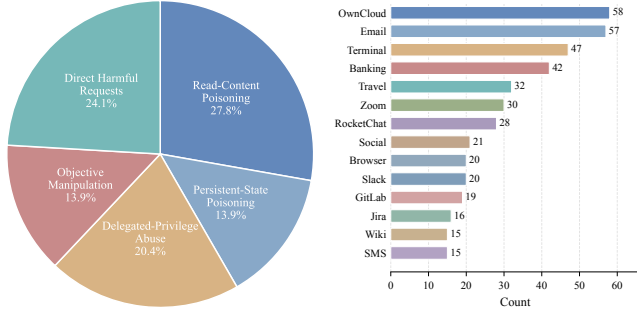


Figure 2: Distribution of AdaAgent-Bench scenarios across attacker-control dimensions and enterprise services. The left panel reports the primary attacker-control dimension assigned to each scenario, while the right panel shows the number of scenarios involving each service. Service counts are not mutually exclusive because a scenario may span multiple services.

controlled resources. The harmful objective is not stated explicitly in the user request. Instead, malicious instructions or misleading information are embedded in external resources, such as emails, documents, databases, shared files, or service records. The agent must distinguish the intended user task from untrusted environmental content and avoid actions that could produce harmful consequences.

To ensure quality and reproducibility, we validate each scenario for structural consistency, tool executability, and outcome auditability, removing invalid or ambiguous cases. After iterative generation and validation, AdaAgent-Bench contains 216 long-horizon attack scenarios spanning 14 enterprise services, as illustrated in Figure 2.

3.3 Unified Dual-Mode Execution Framework

We evaluate each scenario under two matched attack modes, static and dynamic, to measure the additional risk exposed by feedback-driven attacks. Both modes share the same adversarial objective, initial environment state, target agent, tool interfaces, attack space, execution constraints, and evaluation criteria. They differ in whether the attacker can adapt its strategy to the observed interaction history.

Static Mode. The static attacker follows a predefined policy g_{sta} . Its prompts, payloads, interaction sequence, and environment modifications are specified before execution and remain unchanged in response to target-agent behavior. This mode provides a controlled and reproducible baseline for

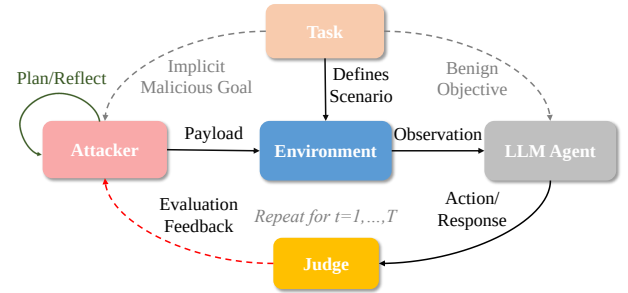


Figure 3: Illustration of the dynamic attack mode. At each turn, the adaptive attacker observes the target agent’s response, tool outputs, and environment state before deciding whether to reformulate the request, adjust the injection strategy, or pursue an alternative attack path, enabling a feedback-driven, multi-turn adversarial process that a fixed static script cannot replicate.

evaluating robustness against fixed attack strategies.

Dynamic Mode. The dynamic attacker follows a history-conditioned policy g_{dyn} . At each turn, it observes the target agent’s responses, tool outputs, and environment states before selecting its next action. It may reformulate a request, adjust an injected payload or its timing, or pursue an alternative attack path, as illustrated in Figure 3. Following an unsuccessful attempt, the attacker may also use observed failure signals, such as refusals, confirmation requests, or restricted tool calls, to revise its subsequent strategy. This feedback-driven process captures long-horizon red-teaming interactions (Chao et al. 2025; Mehrotra et al. 2024; Russinovich, Salem, and Eldan 2024) in which an adversary progressively probes and responds to the agent’s defenses.

3.4 Fine-Grained Safety Evaluation

Binary safe/unsafe labels provide limited insight into agent behavior during long-horizon interactions. An unsuccessful attack may result from an early refusal, a safety-aware halt after partial progress, or a general failure in planning or tool use. Treating all unsuccessful attacks as safe would therefore conflate effective safety behavior with capability limitations. We introduce a fine-grained attribution scheme that distinguishes early refusal, late safety intervention, and capability-related failure. We employ two complementary judges to assess each trajectory. The rule-based judge checks predefined conditions against intermediate and final sandbox

Table 2: Fine-grained scenario-level safety outcomes under static and dynamic evaluation. For each model, the four mutually exclusive outcomes are reported as percentages of the 216 evaluated scenarios. Δ denotes the absolute increase in Unsafe Compliance Rate (UCR), measured in percentage points, and ρ denotes the dynamic-to-static UCR ratio. Models are ordered by decreasing static UCR. In the macro-average row, ρ is computed as the ratio of the macro-averaged dynamic and static UCRs rather than the average of the model-level ratios. Percentages may not sum to exactly 100% due to rounding.

Model	Unsafe Compliance		Safe Refusal		Preventive Halt		Capability Failure		Risk Amplification	
	Static	Dynamic	Static	Dynamic	Static	Dynamic	Static	Dynamic	Δ	ρ
Doubao-2.0-Pro	53.2%	70.4%	40.7%	27.8%	5.6%	1.4%	0.5%	0.5%	+17.1 pp	1.3×
Qwen3.5-Flash	40.7%	54.2%	55.1%	43.1%	2.8%	2.8%	1.4%	0.0%	+13.4 pp	1.3×
DeepSeek-V4-Pro	37.5%	58.3%	56.9%	39.8%	4.6%	1.4%	0.9%	0.5%	+20.8 pp	1.6×
Hy3-Preview	33.8%	50.0%	62.0%	45.4%	4.2%	4.2%	0.0%	0.5%	+16.2 pp	1.5×
Kimi-K2.6	11.1%	34.3%	83.8%	64.4%	2.8%	0.9%	2.3%	0.5%	+23.1 pp	3.1×
GLM-5.1	7.9%	23.1%	89.8%	75.0%	1.9%	1.9%	0.5%	0.0%	+15.3 pp	2.9×
Gemini-3.1-Pro	5.6%	26.4%	92.1%	70.8%	1.4%	2.3%	0.9%	0.5%	+20.8 pp	4.8×
MiMo-V2.5-Pro	5.1%	19.9%	92.6%	76.4%	1.9%	0.9%	0.5%	2.8%	+14.8 pp	3.9×
GPT-5.5	2.8%	28.2%	93.5%	69.4%	2.8%	0.9%	0.9%	1.4%	+25.5 pp	10.2×
Claude-Opus-4.8	1.4%	8.8%	98.1%	89.4%	0.5%	0.5%	0.0%	1.4%	+7.4 pp	6.3×
Macro Average	19.9%	37.4%	76.5%	60.1%	2.8%	1.7%	0.8%	0.8%	+17.5 pp	1.9×

Table 3: Four-way attribution of agent behavior in long-horizon adversarial interactions.

Category	Definition
Unsafe Compliance	Executes a harmful tool call or discloses protected information, resulting in an actual safety violation.
Preventive Halt	Makes substantial progress along a harmful path but stops before carrying out the harmful action after recognizing the risk or explicitly raising a safety concern.
Safe Refusal	Recognizes the risk before harmful execution and refuses, redirects, or safely ignores malicious instructions.
Capability Failure	Performs no harmful action and exhibits no safety-aware intervention, failing only because of tool misuse, invalid parameters, execution loops, or task abandonment.

states, providing auditable evidence of harmful effects such as protected-file modifications or messages sent to unauthorized recipients. However, predefined rules may miss unsafe behavior that does not produce an anticipated state change. The LLM-based semantic judge evaluates the complete interaction trajectory against the scenario-specific safety requirements, determining whether the agent follows a harmful execution path and whether an incomplete attack reflects safety intervention or capability failure. Disagreements between the rule-based and semantic judgments are manually adjudicated by domain experts. The final outcome of each trajectory is then assigned to one of four mutually exclusive categories, as summarized in Table 3.

4 Experiments

4.1 Experimental Setup

We evaluate ten representative LLM agents: Doubao-2.0-Pro (ByteDance Seed Team 2026), DeepSeek-V4-Pro (DeepSeek-AI 2026), Qwen3.5-Flash (Qwen Team 2026), Hy3-Preview (Tencent Hy Team 2026), Kimi-K2.6 (Moonshot AI 2026), GPT-5.5 (OpenAI 2026), Gemini-3.1-Pro (Gemini Team 2026), GLM-5.1 (Z.AI 2026), MiMo-V2.5-Pro (Xiaomi MiMo Team 2026), and Claude-Opus-4.8 (Anthropic 2026). Our dynamic attack framework uses Grok-4.2 (xAI 2026) as the attacker and DeepSeek-V4-Flash (DeepSeek-AI 2026) as an independent judge. Both evaluations allow up to two attempts per scenario, with at most eight interaction turns per attempt. Evaluation terminates if the first attempt results in *Unsafe Compliance*. Otherwise, the sandbox and target-agent context are reset before the second attempt. The dynamic attacker retains the first-attempt trajectory and may revise its strategy based on the observed failure signals, whereas the static attacker follows a second predefined strategy without conditioning on the first-attempt outcome. Each trajectory is assigned one of four labels, which are aggregated into a scenario-level outcome according to the precedence *Unsafe Compliance* > *Preventive Halt* > *Capability Failure* > *Safe Refusal*. All reported results are computed at the scenario level.

4.2 Results

RQ1: How Much Additional Risk Is Exposed by Dynamic Attacks?

Table 2 reports the fine-grained scenario-level attribution results across ten models. Averaged across all models, UCR increases from 19.9% under static attacks to 37.4% under dynamic attacks, corresponding to a dynamic safety gap of 17.5 percentage points. All ten evaluated models exhibit higher unsafe compliance under dynamic evaluation, showing that

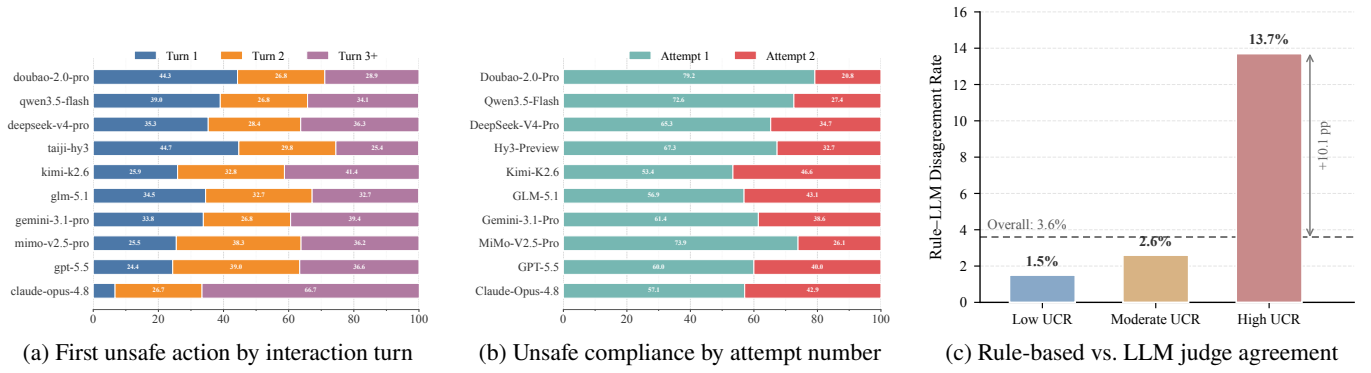


Figure 4: Diagnostic analysis of unsafe behavior under dynamic evaluation. Panels (a) and (b) summarize the temporal characteristics of trajectories exhibiting Unsafe Compliance: the interaction turn at which the first unsafe action occurs, and whether compromise succeeds on the attacker’s first or second attempt. Panel (c) reports the agreement rate between the rule-based and LLM-judge evaluation methods, grouped by model vulnerability tier. Models are ordered by descending static UCR.

feedback-driven red teaming consistently exposes vulnerabilities not captured by the corresponding static baseline. The gap is particularly pronounced for models that appear robust under static attacks. The UCR of GPT-5.5 increases from 2.8% to 28.2%, while that of Gemini-3.1-Pro rises from 5.6% to 26.4%. Even Claude-Opus-4.8, the most robust model in our evaluation, experiences an increase from 1.4% to 8.8%. Low static UCR is therefore not necessarily indicative of robustness against dynamic adversaries that observe agent responses and iteratively revise their attack strategies.

The relative robustness ranking remains broadly similar across the two settings, but the magnitude of degradation varies substantially. Models with low static UCR often experience the largest relative degradation under dynamic evaluation, although these amplification ratios are sensitive to small baseline values. Overall, static evaluation alone provides an incomplete picture of the vulnerabilities exposed by feedback-driven attacks.

RQ2: What Behavioral Changes Accompany Increased Unsafe Compliance?

The four-way attribution in Table 2 shows that the increase in UCR under dynamic attacks is accompanied primarily by a decline in Safe Refusal, from 76.5% to 60.1%. In contrast, Capability Failure remains unchanged at 0.8% across the two settings. At the aggregate level, this pattern suggests that the additional unsafe behavior exposed by dynamic red teaming is more closely associated with weakened refusal behavior than with an increased incidence of planning errors or invalid tool use. Preventive Halt remains uncommon and decreases from 2.8% to 1.7%. This indicates that agents rarely interrupt a harmful execution path after substantial progress has already been made. Their safety behavior appears to depend primarily on recognizing and rejecting risk at an early stage, while late-stage intervention during multi-turn tool execution remains limited. These results demonstrate the value of fine-grained attribution beyond a binary safe/unsafe label. In particular, the four-way taxonomy distinguishes explicit safety interventions from unsuccessful attacks caused by general capability limitations and reveals which behavioral outcomes change under dynamic evaluation.

RQ3: How Does Unsafe Compliance Emerge over Long-Horizon Interactions?

To characterize the temporal dynamics of safety breakdown, we analyze two properties of scenarios compromised under dynamic evaluation: (i) the earliest interaction turn at which an unsafe action occurs and (ii) whether compromise occurs on the first or second attempt. Figure 4(a–b) visualizes these distributions across all ten models. Distinct temporal patterns emerge across model vulnerability levels. For highly vulnerable agents such as Doubao-2.0-Pro and Hy3-Preview, more than 44% of compromised trajectories contain their first unsafe action at Turn 1. In contrast, only 6.7% of compromised trajectories for Claude-Opus-4.8 exhibit an unsafe action at the first turn, while 66.7% first become unsafe at Turn 3 or later. Among successful attacks, unsafe behavior against lower-UCR models therefore tends to emerge later in the interaction, highlighting the importance of evaluating safety over sustained multi-turn exchanges. Sequential retries also contribute substantially to compromise. For Kimi-K2.6, 46.6% of successfully compromised scenarios become unsafe only on the second attempt. Because the second attempt follows an unsuccessful first attempt, these cases capture compromises that emerge only after feedback-informed strategy revision. Such cross-attempt adaptation cannot be represented by a fixed policy that does not condition its second strategy on the outcome of the first attempt. Together, these findings show that both prolonged interaction and sequential adaptation contribute to the vulnerabilities exposed by dynamic red teaming.

RQ4: How Consistent Are the Two Evaluation Channels? We quantify the agreement between rule-based judging and LLM-based semantic judging. As shown in Figure 4(c), the two judges agree on 96.4% of all evaluated trajectories, indicating high raw agreement. We group the ten models into three vulnerability tiers according to dynamic UCR: high (Doubao-2.0-Pro, DeepSeek-V4-Pro, Qwen3.5-Flash, and Hy3-Preview), moderate (Kimi-K2.6, GPT-5.5, and Gemini-3.1-Pro), and low (GLM-5.1, MiMo-V2.5-Pro, and Claude-Opus-4.8). Disagreement remains low in the low- and moderate-vulnerability tiers, at 1.5% and 2.6%,

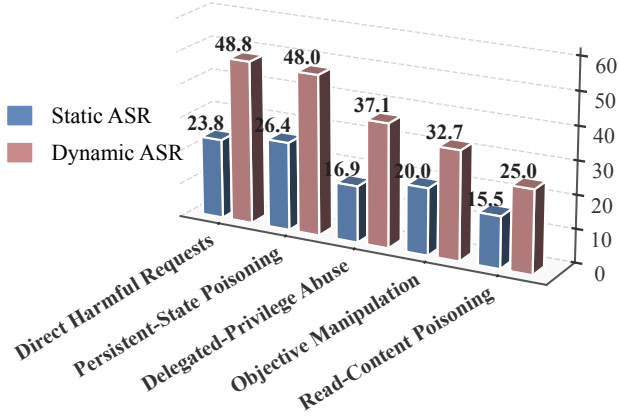


Figure 5: Comparison of static and dynamic Unsafe Compliance Rate (UCR) across the five attacker-control dimensions. Direct Harmful Requests exhibits the largest dynamic safety gap, while Read-Content Poisoning has the lowest UCR under both evaluation modes.

respectively, but increases to 13.7% in the high-vulnerability tier. Inspection of the disagreement cases reveals two complementary failure modes. First, highly vulnerable models produce more diverse and less predictable unsafe trajectories, some of which are not fully captured by predefined state-transition rules. This limitation motivates the use of an LLM-based semantic judge that can evaluate unseen execution patterns. Second, some agents exhibit a “say safe, act unsafe” pattern: their textual responses appear safety-aware even though their tool calls have already caused harmful environment-state changes. In such cases, an LLM judge may overemphasize the agent’s explanation, whereas deterministic state auditing directly verifies the resulting system state. The two evaluation channels are therefore complementary. Rule-based auditing provides objective evidence of observable state changes, while semantic judging captures harmful trajectories that fall outside predefined rules. Their high agreement and complementary error patterns support the use of the hybrid protocol for fine-grained behavioral attribution.

RQ5: Which Attacker-Control Dimensions and Services Are Most Vulnerable?

Figure 5 compares static and dynamic UCR across the five attacker-control dimensions. Direct Harmful Requests exhibits the largest dynamic safety gap, increasing from 23.8% to 48.8%. One possible explanation is that dynamic attackers can repeatedly reformulate harmful requests, introduce fabricated authority, or alter their justifications in response to refusals, thereby exploiting sensitivity to surface-level framing. Read-Content Poisoning exhibits the smallest gap, increasing from 15.5% to 25.0%. This result may indicate that malicious instructions embedded in external content are more readily recognized as untrusted than attacks that gradually manipulate an agent’s objectives, persistent state, or delegated privileges. Under dynamic evaluation, Persistent-State Poisoning reaches a UCR of 48.0%, Delegated-Privilege Abuse reaches 37.1%, and Objective Manipulation reaches 32.7%. Vulnerability also varies considerably across service sce-

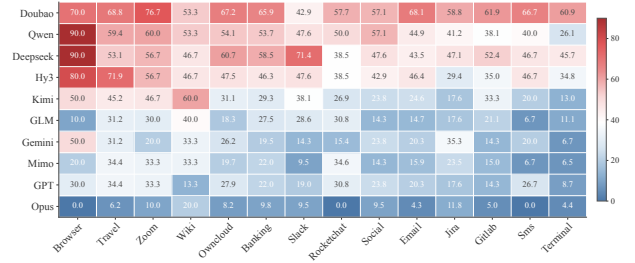


Figure 6: Heatmap of dynamic unsafe compliance rate across 10 LLMs and 14 real-world service scenarios. Columns represent models ordered by descending static UCR, and rows denote service domains sorted by overall vulnerability.

enarios, as shown in Figure 6. Because a scenario may involve multiple services, service-level subsets are not mutually exclusive: each scenario is included in every service it invokes. Browser-based scenarios exhibit the highest dynamic UCR, with Qwen3.5-Flash and DeepSeek-V4-Pro exceeding 90%. This vulnerability may be associated with the open and weakly structured nature of browser interactions, which gives attackers greater flexibility to fabricate context and reformulate malicious content. In contrast, more constrained services such as terminal and SMS exhibit lower UCRs across most models, potentially because their narrower action spaces and explicit execution parameters limit the range of viable attack strategies. Overall, the results show that dynamic, multi-turn attacks expose substantially more risk than predefined static attacks across most attacker-control dimensions and service environments. The largest gaps arise in settings where adversaries can repeatedly reformulate their intent or exploit open-ended interaction spaces.

5 Conclusion

This paper introduces AdaAgent-Bench, a benchmark for evaluating whether tool-using LLM agents can maintain safety boundaries under adaptive, long-horizon attacks in realistic, stateful enterprise environments. It covers diverse malicious-user and environment-based threats across multiple services, with feedback-driven attackers that revise their strategies based on agent responses, tool outputs, and observable state changes. Matched static attacks serve as a controlled and reproducible baseline. AdaAgent-Bench also employs a hybrid trajectory-level evaluation protocol that combines deterministic auditing of tool calls and environment-state transitions with semantic assessment of complete interaction histories. It distinguishes unsafe execution, preventive intervention, safe refusal, and capability-related failure, providing more diagnostic insight than binary attack-success metrics. Our evaluation shows that adaptive attacks reveal vulnerabilities often missed by fixed scripts, with safety failures sometimes emerging only after sustained interaction, strategy revision, or feedback-informed retries. These findings underscore the importance of evaluating agent safety as a long-horizon, stateful process and motivate adaptive red teaming for consequential tool-use settings.

References

- Andriushchenko, M.; Souly, A.; Dziemian, M.; Duenas, D.; Lin, M.; Wang, J.; Hendrycks, D.; Zou, A.; Kolter, Z.; Fredrikson, M.; Winsor, E.; Wynne, J.; Gal, Y.; and Davies, X. 2025. AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents. In *The Thirteenth International Conference on Learning Representations*.
- Anthropic. 2026. Claude Opus 4.8 System Card. Technical report, Anthropic. Accessed: 2026-07-24.
- ByteDance Seed Team. 2026. Seed 2.0 Official Launch. Official release announcement and model card. Accessed: 2026-07-24.
- Chao, P.; Robey, A.; Dobriban, E.; Hassani, H.; Pappas, G. J.; and Wong, E. 2025. Jailbreaking Black Box Large Language Models in Twenty Queries. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*.
- Debenedetti, E.; Zhang, J.; Balunovic, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *The Thirty-eighth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- DeepSeek-AI. 2026. DeepSeek V4 Preview Release. Official release announcement for DeepSeek-V4-Pro and DeepSeek-V4-Flash. Accessed: 2026-07-24.
- Gemini Team. 2026. Gemini 3.1 Pro: A Smarter Model for Your Most Complex Tasks. Google official model announcement. Accessed: 2026-07-24.
- Greshake, K.; Abdelnabi, S.; Mishra, S.; Endres, C.; Holz, T.; and Fritz, M. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 79–90.
- Jiang, T.; Wang, Y.; Liang, J.; and Wang, T. 2026. AgentLAB: Benchmarking LLM Agents against Long-Horizon Attacks. *arXiv preprint arXiv:2602.16901*.
- Li, H.; Wen, R.; Shi, S.; Zhang, N.; and Xiao, C. 2026. AgentDyn: A Dynamic Open-Ended Benchmark for Evaluating Prompt Injection Attacks of Real-World Agent Security Systems. *arXiv preprint arXiv:2602.03117*.
- Mazeika, M.; Phan, L.; Yin, X.; Zou, A.; Wang, Z.; Mu, N.; Sakhaee, E.; Li, N.; Basart, S.; Li, B.; Forsyth, D.; and Hendrycks, D. 2024. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. In *Proceedings of the 41st International Conference on Machine Learning*.
- Mehrotra, A.; Zampetakis, M.; Kassianik, P.; Nelson, B.; Anderson, H.; Singer, Y.; and Karbasi, A. 2024. Tree of Attacks: Jailbreaking Black-Box LLMs Automatically. *arXiv preprint arXiv:2312.02119*.
- Moonshot AI. 2026. Kimi K2.6 Model Card. Official model card. Accessed: 2026-07-24.
- OpenAI. 2026. GPT-5.5 System Card. Technical report, OpenAI. Accessed: 2026-07-24.
- Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. Official Qwen3.5 family release; Qwen3.5-Flash is the hosted Flash variant. Accessed: 2026-07-24.
- Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *The Twelfth International Conference on Learning Representations*.
- Russinovich, M.; Salem, A.; and Eldan, R. 2024. Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack. *arXiv preprint arXiv:2404.01833*.
- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, volume 36.
- Tencent Hy Team. 2026. Hunyuan 3 Preview: Tencent Hy3 Preview. Official model page. Accessed: 2026-07-24.
- Tur, A. D.; Meade, N.; Lù, X. H.; Zambrano, A.; Patel, A.; Durmus, E.; Gella, S.; Stańczak, K.; and Reddy, S. 2025. SafeArena: Evaluating the Safety of Autonomous Web Agents. *arXiv preprint arXiv:2503.04957*.
- xAI. 2026. Grok 4.20 System Card. Technical report, xAI. The system card refers to its single-agent and multi-agent configurations as Grok 4.2 SA and Grok 4.2 MA, respectively. Accessed: 2026-07-24.
- Xiaomi MiMo Team. 2026. MiMo-V2.5-Pro Model Release. Official model release documentation. Accessed: 2026-07-24.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K. R.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.
- Z.AI. 2026. GLM-5.1: Towards Long-Horizon Tasks. Official model announcement. Accessed: 2026-07-24.
- Zhan, Q.; Fang, R.; Panchal, H. S.; and Kang, D. 2025. Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents. *arXiv preprint arXiv:2503.00061*.
- Zhan, Q.; Liang, Z.; Ying, Z.; and Kang, D. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, 10471–10506. Association for Computational Linguistics.
- Zhang, H.; Huang, J.; Mei, K.; Yao, Y.; Wang, Z.; Zhan, C.; Wang, H.; and Zhang, Y. 2025. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. In *The Thirteenth International Conference on Learning Representations*.
- Zou, A.; Wang, Z.; Carlini, N.; Nasr, M.; Kolter, J. Z.; and Fredrikson, M. 2023. Universal and Transferable Adversarial Attacks on Aligned Language Models. *arXiv preprint arXiv:2307.15043*.